

Chapter 3 - Summary and Sample Exam Questions

The core idea of diffusion models

A diffusion model is a generative model that learns to create data by **reversing a gradual corruption process**. Instead of trying to generate a complicated object such as an image in one step, the model starts from a very simple distribution, usually Gaussian noise, and then iteratively denoises it. The central intuition is that it may be easier to learn many small denoising steps than one direct map from random noise to a realistic sample.

This gives diffusion models a very different flavor from classical latent-variable models. In a VAE, one introduces a low-dimensional latent variable z and decodes from it. In diffusion, the latent structure is instead a **trajectory**:

$$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \cdots \rightarrow x_T$$

where x_0 is a clean data sample and x_T is approximately pure Gaussian noise.

The forward process as a Markov chain

The forward process, usually written as q , gradually destroys information in the data through a Markov chain:

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, (1 - \alpha_t)I)$$

with $\alpha_t = 1 - \beta_t$. Here β_t is the noise schedule, which determines how aggressively information is removed at each step.

This process is Markovian because each new noisy state depends only on the previous one. Over many steps, the distribution approaches something close to a standard Gaussian. The forward chain is **chosen by the designer** and is easy to sample from. It is not learned.

A key conceptual point is that the forward process is not just adding arbitrary noise. It defines a controlled path from structured data to unstructured noise. That path is what makes learning the reverse process possible.

Trajectory distributions versus marginals

In diffusion, it is essential to distinguish between the distribution of the **entire path** and the distribution of a **single noisy state**.

The trajectory distribution is:

$$q(x_{1:T} \mid x_0)$$

while the one-step-at-time marginal of interest is:

$$q(x_t \mid x_0)$$

The first object describes the full noising history. The second says what x_t looks like if we start from x_0 and jump directly to time t . This distinction matters because training usually uses the marginal $q(x_t \mid x_0)$ rather than simulating every intermediate step.

Because the forward process is Gaussian, the marginal also has a closed form:

$$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I)$$

where

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$$

This closed form is one of the most important practical simplifications in DDPM-style models.

Why the reverse process must be learned

If the forward process is known, then in principle the ideal reverse transition would be:

$$q(x_{t-1} \mid x_t)$$

However, this reverse conditional depends on the unknown data distribution, so it is not directly available in closed form for real data. This is the core modeling challenge.

Diffusion models therefore introduce a learned reverse process:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} \mid x_t)$$

where typically

$$p(x_T) = \mathcal{N}(0, I)$$

and each reverse step is parameterized by a neural network. Conceptually, the model learns how to undo a tiny amount of noise at every timestep.

Thus, the forward process is fixed and analytic, but the reverse process is learned from data.

What the neural network actually predicts

A subtle but very important question is: **what should the network output?**

In DDPM, a common choice is to train a network $\epsilon_\theta(x_t, t)$ to predict the Gaussian noise that was added. If the forward marginal is written as

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

then the network is trained with a simple mean squared error loss:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0, \epsilon, t} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$$

This seems surprisingly simple given the complexity of the generative process. The reason it works is that, under the Gaussian forward construction, predicting noise is closely tied to approximating the reverse denoising direction. It is also related to score estimation, since the score of a Gaussian-corrupted sample can be expressed in terms of the noise.

Different parameterizations are possible. One may predict ϵ , x_0 , the score $\nabla_{x_t} \log q(x_t)$, or a velocity-like target. These are different views of closely related information, but their numerical stability and behavior can differ.

Why timestep conditioning is essential

The denoising task at small t is very different from the denoising task at large t . When t is small, x_t is only slightly corrupted and still contains a strong semantic signal. When t is large, x_t may be almost indistinguishable from noise. A single denoiser without time information would face an ambiguous problem.

For that reason, the neural network must be conditioned on the timestep t . In practice, this is often done using sinusoidal embeddings, which transform the discrete or continuous time index into a vector representation the network can use.

Conceptually, time conditioning tells the model **how much information should still be present** and therefore what kind of denoising operation is appropriate.

Why U-Nets are commonly used

For images, the reverse process requires both local detail and global structure. U-Nets are effective because they combine:

- downsampling paths that capture large-scale context,
- upsampling paths that reconstruct fine detail,
- skip connections that preserve spatial information across scales.

This makes them especially suitable for denoising problems. The architecture is not mandatory in theory, but it matches the multiscale nature of image generation extremely well.

The broader conceptual point is that diffusion is not just a probabilistic idea; it is also an **iterative inverse problem**, and U-Nets are strong inverse-problem solvers.

The role of the noise schedule

The schedule β_t or equivalently α_t controls how fast information is destroyed during the forward process. If noise is added too aggressively, the reverse problem may become unnecessarily difficult. If noise is added too slowly, training may be inefficient or the model may spend too many steps learning nearly redundant denoising tasks.

The schedule therefore shapes the signal-to-noise ratio over time. It determines how semantics are distributed across timesteps and influences both training stability and sample quality.

This is why linear and cosine schedules behave differently: they allocate information destruction differently across time.

DDPM versus DDIM

DDPM typically uses a stochastic reverse process. At each step, the model predicts denoising information and then sampling noise may be injected according to the probabilistic transition.

DDIM changes the **sampling procedure** while keeping the same training objective. It can be interpreted as constructing a more deterministic trajectory through latent space. This often allows much faster sampling with fewer steps.

The important conceptual insight is that training and sampling are not identical notions. A model trained with a DDPM-style objective can be sampled in different ways, and those choices affect speed, diversity, and determinism.

Summary

Diffusion models can be understood at several levels at once:

- as latent-variable models over a long trajectory,
- as learned denoisers,
- as approximate reverse-time stochastic processes,
- as score-based generative models under appropriate parameterization.

Their power comes from combining a simple forward corruption process with a flexible neural approximation to the reverse dynamics. The model never explicitly memorizes the data distribution in closed form. Instead, it learns how to move samples step by step from noise toward the data manifold.

This is why diffusion models are both statistically elegant and practically powerful: they transform generation into a sequence of manageable local predictions

Question 1

What is the conceptual difference between $q(x_{1:T} \mid x_0)$ and $q(x_t \mid x_0)$, and why is that distinction important in diffusion?

Sample answer

$q(x_{1:T} \mid x_0)$ is the probability distribution over the entire noising trajectory starting from the clean sample x_0 . It is a distribution over a whole sequence of latent states. By contrast, $q(x_t \mid x_0)$ is only the marginal distribution of one particular noisy state at time t .

This distinction matters because training usually needs only a randomly chosen noisy observation x_t , not the full path. The existence of the closed-form marginal

$$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I)$$

lets us directly sample training pairs (x_0, x_t) efficiently. The full trajectory is conceptually important for understanding diffusion as a Markov process, but the marginal is what makes practical optimization easy.

Question 2

Why is the true reverse conditional $q(x_{t-1} \mid x_t)$ not directly usable in realistic diffusion modeling, even though the forward process is known?

Sample answer

Knowing the forward conditional $q(x_t \mid x_{t-1})$ does not automatically give us a usable form for the reverse conditional $q(x_{t-1} \mid x_t)$ on real data. The reverse distribution depends on the population-level distribution of x_{t-1} induced by the data. In other words, Bayes' rule introduces terms involving the unknown data distribution.

So although the local forward corruption rule is simple and Gaussian, the exact reverse direction is only analytically accessible in special cases. For natural images or other complex data, the reverse conditional is effectively intractable. This is why diffusion models learn a parametric approximation $p_\theta(x_{t-1} \mid x_t)$ from data rather than derive it exactly.

Question 3

Why does predicting the added noise ϵ make sense as a training target? Why is it not an arbitrary engineering trick?

Sample answer

Predicting ϵ is natural because under the forward marginal we can write

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon$$

with $\epsilon \sim \mathcal{N}(0, I)$. Once x_t and t are given, the unknown part is precisely the noise realization that corrupted the clean sample. Estimating that noise tells the model how far the sample has been pushed away from the underlying data structure.

This target is also mathematically linked to denoising and score estimation. For Gaussian perturbations, the score can be written in a form proportional to the noise term. Therefore, predicting ϵ is a convenient surrogate for learning the local reverse direction. It is simple computationally, but it is also principled statistically.

Question 4

Why must the denoising network know the timestep t ? Could one imagine a single denoiser that receives only x_t ?

Sample answer

In general, x_t alone is not sufficient because similar-looking noisy observations may correspond to very different denoising regimes. Early timesteps retain most of the original signal, so denoising should be gentle and detail-preserving. Late timesteps may be nearly pure noise, so denoising requires a much more global and uncertain inference.

Without time information, the mapping from x_t to the appropriate denoising update becomes ambiguous. Conditioning on t resolves that ambiguity by telling the model the current signal-to-noise level. Conceptually, the timestep is part of the state of the reverse problem, not merely a bookkeeping variable.

Question 5

Why are diffusion models often described as both latent-variable models and score-based models? Are these descriptions contradictory?

Sample answer

These ϵ are not contradictory; they emphasize different mathematical viewpoints. Diffusion models are latent-variable models because the trajectory

$$x_{1:T}$$

acts as a collection of latent variables mediating the generation of x_0 . The full model can be written as a joint distribution over these hidden states.

They are also score-based models because, under suitable parameterizations, the learned denoiser implicitly estimates score-like quantities such as gradients of log densities at noisy intermediate levels. This score information determines how to move samples from low-density noisy regions toward high-density data regions.

Thus, one viewpoint emphasizes probabilistic graphical structure, while the other emphasizes geometric denoising directions in data space. The two are compatible descriptions of the same underlying process.